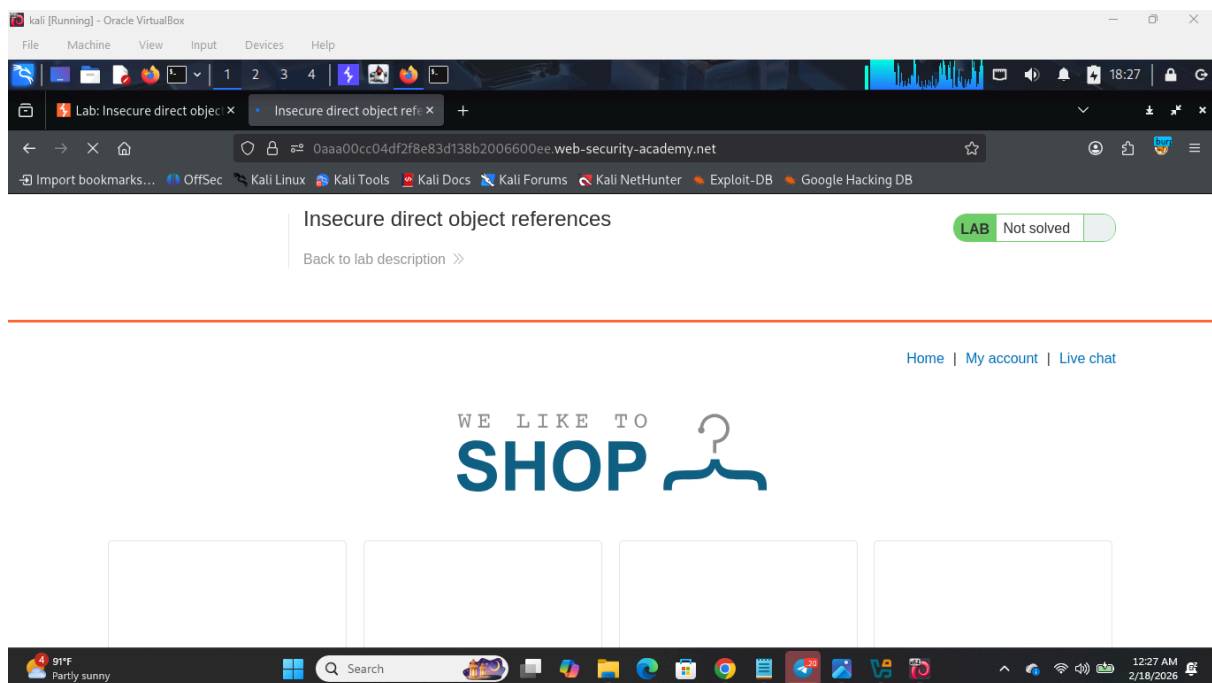


VA INITIATIVES ETHICAL HACKING PROJECT

PROJECT 1: Insecure Direct Object References (IDOR)

Lab Setup & Recon

Target URL: <https://0aaa00cc04df2f8e83d138b2006600ee.web-security-academy.net/>



The web application is an online retail store with a Home page where items are listed with prices and their ratings. There is also a "My account" page where users can login and view their accounts.

Finally, there is a "live chat" feature where users can chat with customer care in real time.

Authentication Status: Logged out

Login

Username

Password

Log in

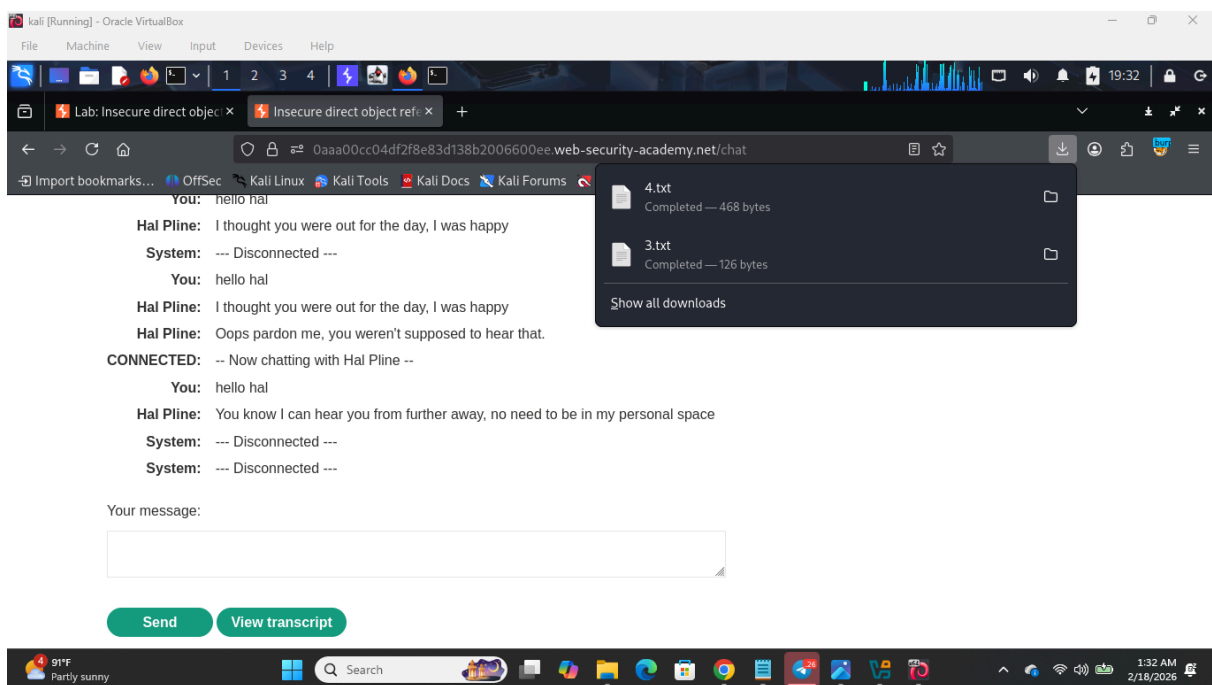
Initial Observed Behavior;

The web application allows unauthenticated user to view listed items and their details.

An unauthenticated user is also allowed to use the "Live Chat" feature of the web app Anonymously.

There is a feature in the "Live chat" page that permits the download of chat history transcript in plain text.

Each transcript is stored serially



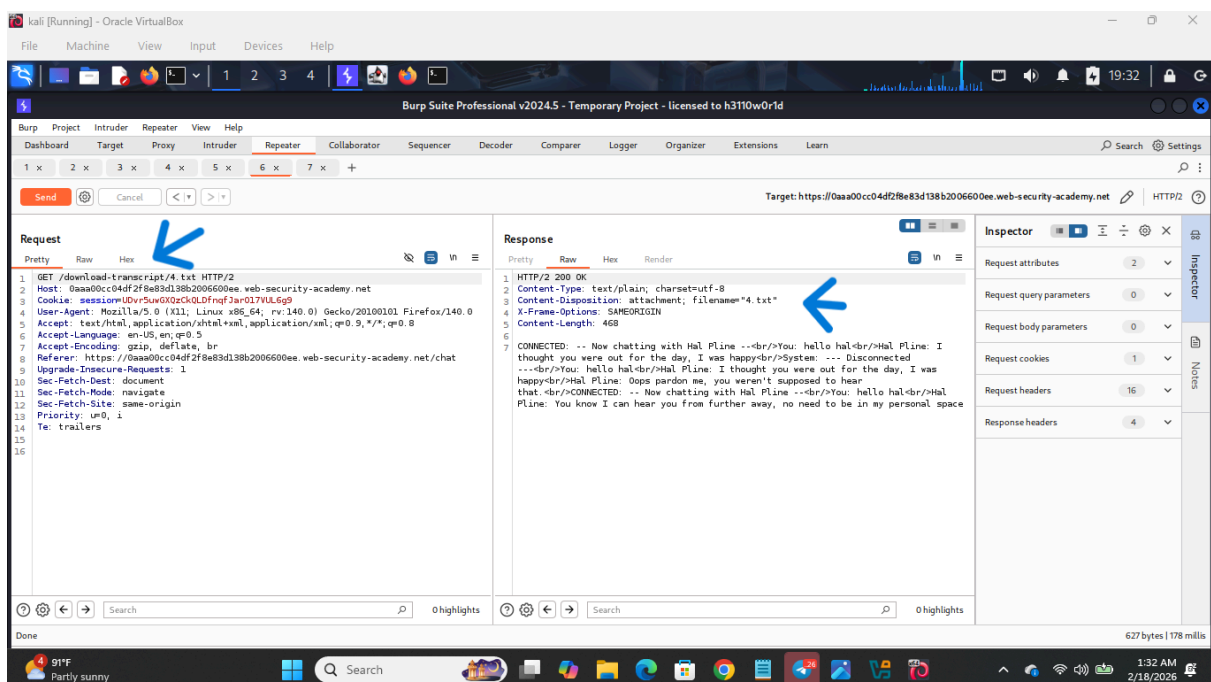
Identifying the Vulnerability

In the previous section, It is observed that as an unauthenticated user, I still have access to the Live chat.

I also observed the transcripts are stored serially.

I captured the transcript download request using my burpsuite.

The URL in the GET request suggests that there are transcript files stored in the "download-transcript" directory in the server. And given the observed naming convention of the transcript files I previously downloaded, It is possible that other transcript files are stored in this directory and are also named using an integer.



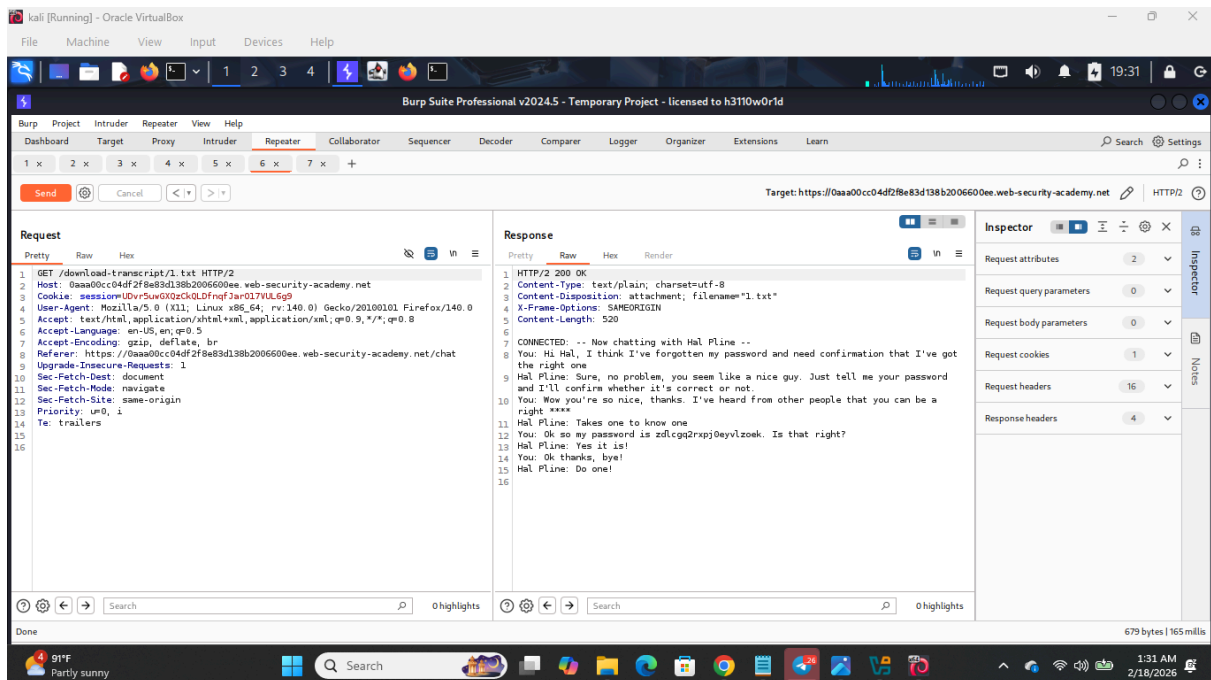
What the app is doing incorrectly;

1. Sequential Filenames: As observed earlier, the app is using Predictable Resource Naming to store transcripts.
2. Direct Access: The app allows request to a file directly from a directory (/download-transcript) without an authorization layer.

3. Plain Text Logs: The server responds by returning requested transcript in plain text.

Exploit the Flaw (PoC)

Modified request and Response showing unauthorized data leak



Why this confirms the vulnerability

This confirms the IDOR vulnerability because when the name of the object in the request "4.txt" was changed to "1.txt"; which belonged to another user and therefore should not be granted access to, the server responded by returning the object and exposing the chat history of another user "carlos" to an unauthorized user.

Assess Impact

The exploitation of this vulnerability exposed the user "carlos" password. An attacker could use that password to gain unauthorized access to the user's account.

Only one user (carlos) was affected.

This impact violates user privacy and goes against compliance policies like the

GDPR (General Data Protection Regulation) and SOC 2 (System and Organization Controls).

Remediation Plan

Issue	fix	Security Benefit
Filename Predictability	Use long, random UUIDs instead of sequential numbers.	Almost impossible to guess
Direct Access:	Checking the user's session cookie before serving the file. Using an indirect reference (an ID in a database that maps to a file).	Only authorized users can access sensitive resources
Plain Text Logs	Encrypt server logs at rest and in transit	protect logs in transit and at rest
Single Authentication	MFA: Mandating a second factor (SMS, App, or Hardware Key) for new session authentication or when logging in on new device.	Adds an extra layer of authentication incase of password leakage.
IDOR	Authorization Checks on sensitive resources. Using encrypted or complex references instead of directly incrementing object IDs.	Secures the system against IDOR vulnerabilities.

References

Burpsuite; Proxy intercept, Repeater

Portswigger IDOR Lab; <https://portswigger.net/web-security/access-control/lab-insecure-direct-object-references>

PROJECT 2: PCAP FILE ANALYSIS

Confirmation that the capture file contains WPA handshake traffic

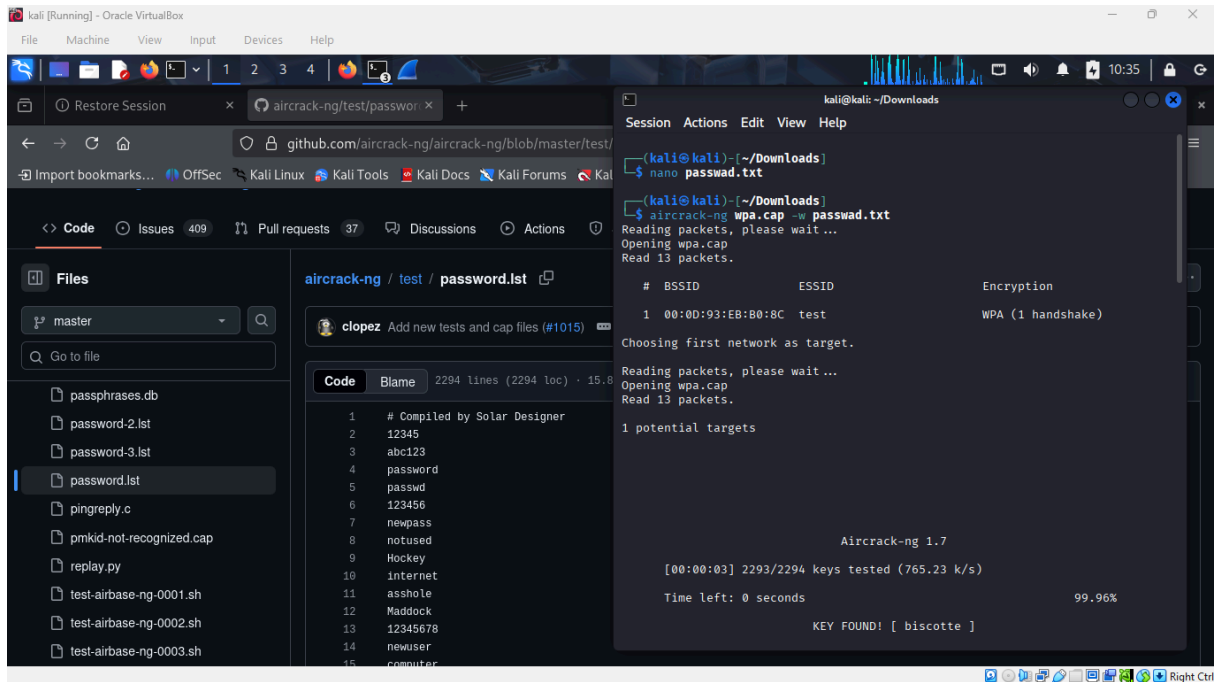
The screenshot shows a Wireshark capture of a WPA handshake. The packet list on the left shows several frames, with frame 8 (279 bytes) selected. The packet details pane on the right shows the structure of the frame, including the IEEE 802.11 Data, Logical-Link Control, and 802.1X Authentication fields. The 802.1X Authentication field is expanded, showing the Key Descriptor Type (EAPOL WPA Key (254)), Key Information (0x0109), and Key Type (Pairwise Key). The packet bytes pane on the right shows the raw data of the frame, with the key information field highlighted in blue.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	Apple_eb:b0:8c	Broadcast	802.11	262	Beacon frame, SN=245, FN=0, Flags=....., BI=100, SSID="test"[Malfo...
2	0.008977	Apple_eb:b0:8c	Netgear_91:53:5d	EAPOL	279	Key (Message 1 of 4)
3	0.009191	Apple_eb:b0:8c	Apple_eb:b0:8c	802.11	158	Acknowledgement, Flags=.....
4	0.071788	Netgear_91:53:5d	Apple_eb:b0:8c	EAPOL	303	Key (Message 2 of 4)
5	0.071995	Netgear_91:53:5d	Netgear_91:53:5d	802.11	158	Acknowledgement, Flags=.....
6	0.074633	Apple_eb:b0:8c	Netgear_91:53:5d	EAPOL	303	Key (Message 3 of 4)
7	0.074847	Apple_eb:b0:8c	Apple_eb:b0:8c	802.11	158	Acknowledgement, Flags=.....
8	0.075765	Netgear_91:53:5d	Apple_eb:b0:8c	EAPOL	279	Key (Message 2 of 4)
9	0.075971	Netgear_91:53:5d	Netgear_91:53:5d	802.11	158	Acknowledgement, Flags=.....
10	0.077038	Apple_eb:b0:8c	Netgear_91:53:5d	802.11	331	Data, SN=248, FN=0, Flags=p....F.
11	0.077247	Apple_eb:b0:8c	Apple_eb:b0:8c	802.11	158	Acknowledgement, Flags=.....
12	0.078402	Netgear_91:53:5d	Apple_eb:b0:8c	802.11	299	Data, SN=915, FN=0, Flags=p.....T
13	0.078607	Netgear_91:53:5d	Netgear_91:53:5d	802.11	158	Acknowledgement, Flags=.....

The screenshot shows a Wireshark capture of a WPA handshake. The packet list on the left shows several frames, with frame 8 (279 bytes) selected. The packet details pane on the right shows the structure of the frame, including the IEEE 802.11 Data, Logical-Link Control, and 802.1X Authentication fields. The 802.1X Authentication field is expanded, showing the Key Descriptor Type (EAPOL WPA Key (254)), Key Information (0x0109), and Key Type (Pairwise Key). The packet bytes pane on the right shows the raw data of the frame, with the key information field highlighted in blue.

No.	Time	Source	Destination	Protocol	Length	Info
2	0.008977	Apple_eb:b0:8c	Netgear_91:53:5d	EAPOL	279	Key (Message 1 of 4)
4	0.071788	Netgear_91:53:5d	Apple_eb:b0:8c	EAPOL	303	Key (Message 2 of 4)
6	0.074633	Apple_eb:b0:8c	Netgear_91:53:5d	EAPOL	303	Key (Message 3 of 4)
8	0.075765	Netgear_91:53:5d	Apple_eb:b0:8c	EAPOL	279	Key (Message 2 of 4)

The recovered Wi-Fi password; "biscotte"



The screenshot shows a Kali Linux virtual machine environment. In the background, a web browser displays the GitHub repository for aircrack-ng. The foreground features a terminal window where the aircrack-ng tool is being used to crack a Wi-Fi password from a capture file named 'wpa.cap'. The tool's output indicates that it has successfully identified the password 'biscotte' after testing 2293 out of 2294 keys.

```
kali@kali: ~/Downloads
$ nano passwd.txt
$ aircrack-ng wpa.cap -w passwd.txt
Reading packets, please wait ...
Opening wpa.cap
Read 13 packets.

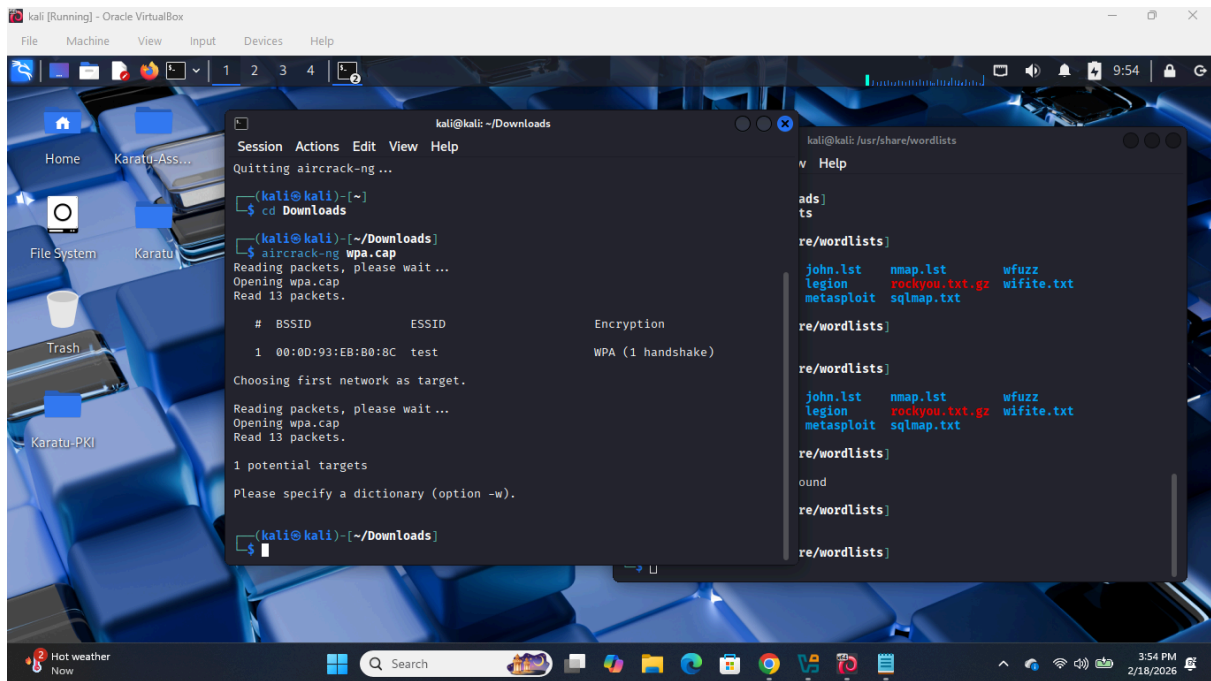
# BSSID      ESSID      Encryption
1 00:00:93:EB:B0:8C test      WPA (1 handshake)

Choosing first network as target.
Reading packets, please wait ...
Opening wpa.cap
Read 13 packets.
1 potential targets

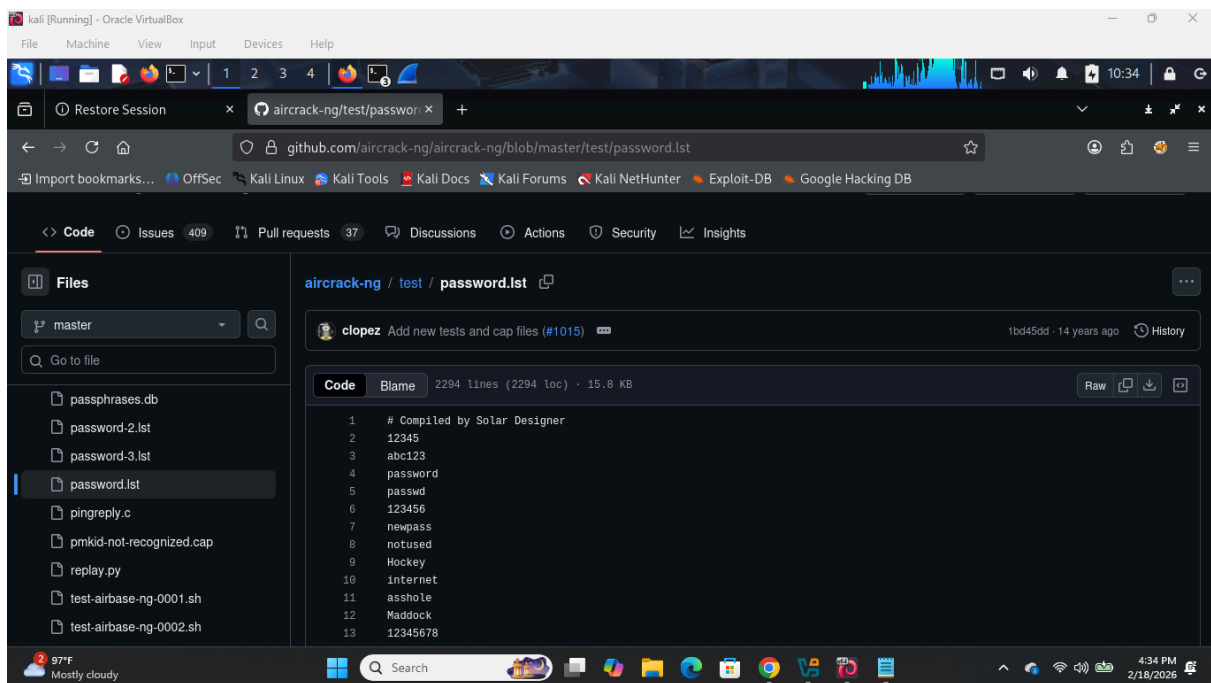
AirCrack-ng 1.7
[00:00:03] 2293/2294 keys tested (765.23 k/s)
Time left: 0 seconds 99.96%
KEY FOUND! [ biscotte ]
```

A brief explanation of the steps you took to recover the password

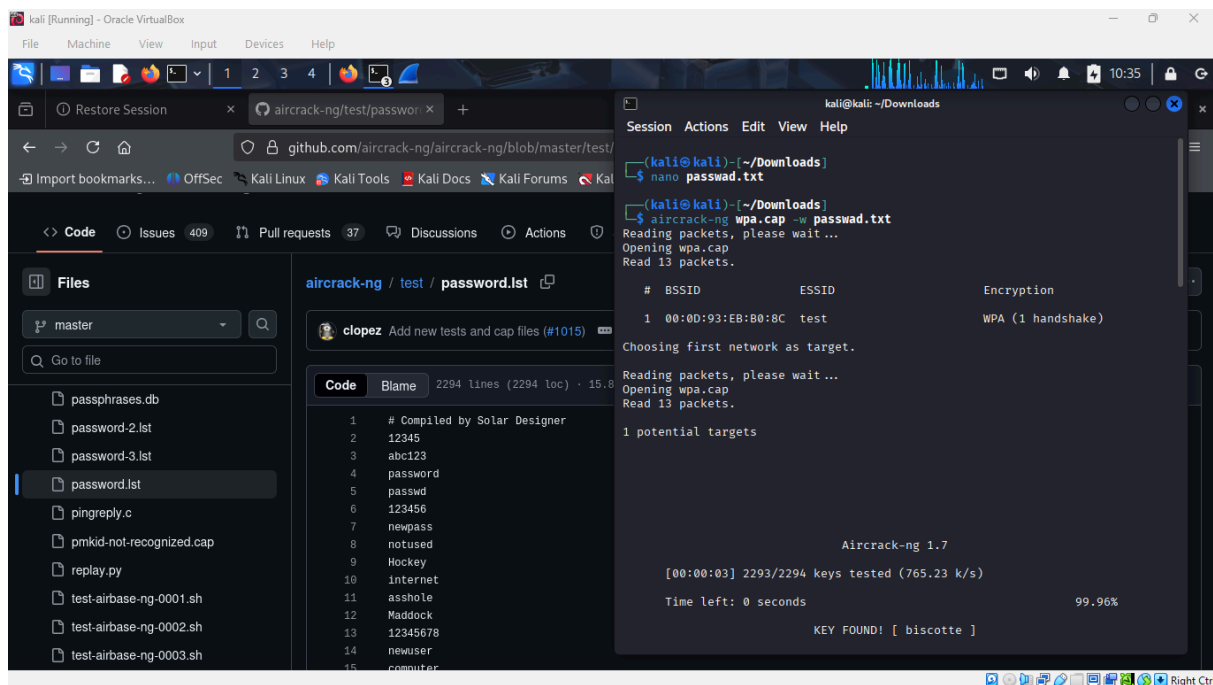
1. I used "aircrack-ng" tool to determine if there is a handshake in the "wpa.cap" file



2. I visited the aircrack-ng github repo, and cloned the password list



3. Using "aircrack-ng wpa.cap -w passwd.txt" command, I ran an offline dictionary attack on the pcap file to recover the password!



A brief explanation of why this password was recoverable

The password for the “**wpa.cap**” file was recoverable because the WPA2-Personal (PSK) protocol has a fundamental design flaw: it uses a **4-way handshake** that transmits enough cryptographic information in plain text for an attacker to perform an **offline dictionary attack**.

Plain Text Handshake: Exposes the variables needed to calculate the MIC (Message Integrity Code).

PBKDF2 Weakness: Predictable passwords can be pre-calculated or cracked quickly.

How stronger passwords or WPA3 would mitigate this risk

Using Stronger Passwords

- Strong passwords use a mix of uppercase, lowercase, numbers, and symbols, which increases the number of combinations an attacker must guess. A complex password like `G7!pQ9z#2m` will not appear in standard wordlists (like the one used to find `biscotte`), making a dictionary attack fail.

- **Computational Cost:** Because WPA2 uses the PBKDF2 function with 4,096 iterations, it is intentionally slow to calculate. A long, complex password could take centuries to crack even with modern hardware.

Using WPA3

- WPA3 replaces the vulnerable 4-way handshake of WPA2 with a new protocol called **Simultaneous Authentication of Equals (SAE)**, also known as the **Dragonfly Handshake**.
- **Password-Authenticated Key Exchange (PAKE):** SAE uses a cryptographic exchange where the device and router prove they know the password without actually sending a hash of it over the air.
- Unlike WPA2, an attacker who captures the WPA3 handshake cannot take it home and run a dictionary attack against it. The captured data is not enough to verify a password guess.
- **Rate Limiting:** If an attacker tries too many incorrect passwords, the router can simply block them, just like a website blocks you after five failed login attempts. This turns an "unlimited" offline attack into a "limited" online attack.

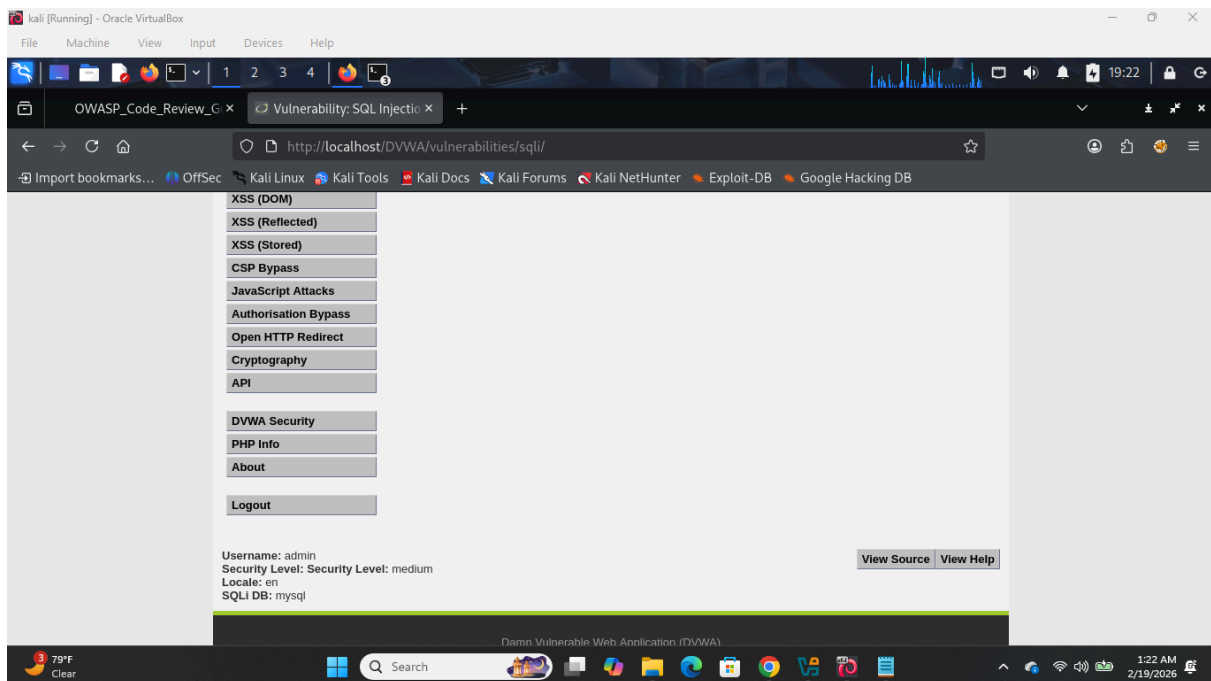
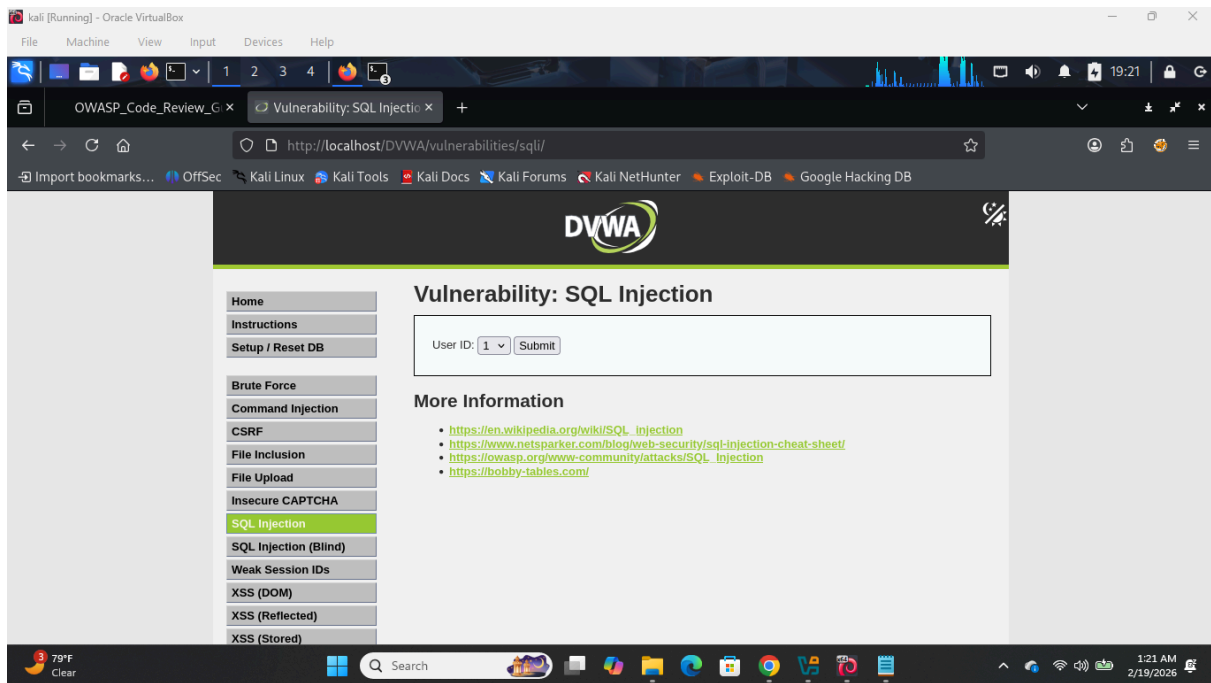
References

Wireshark

aircrack-ng; github and CLI tool

Project 3: SQL Injection at Medium Security Level (DVWA)

Confirmation of DVWA Medium configuration



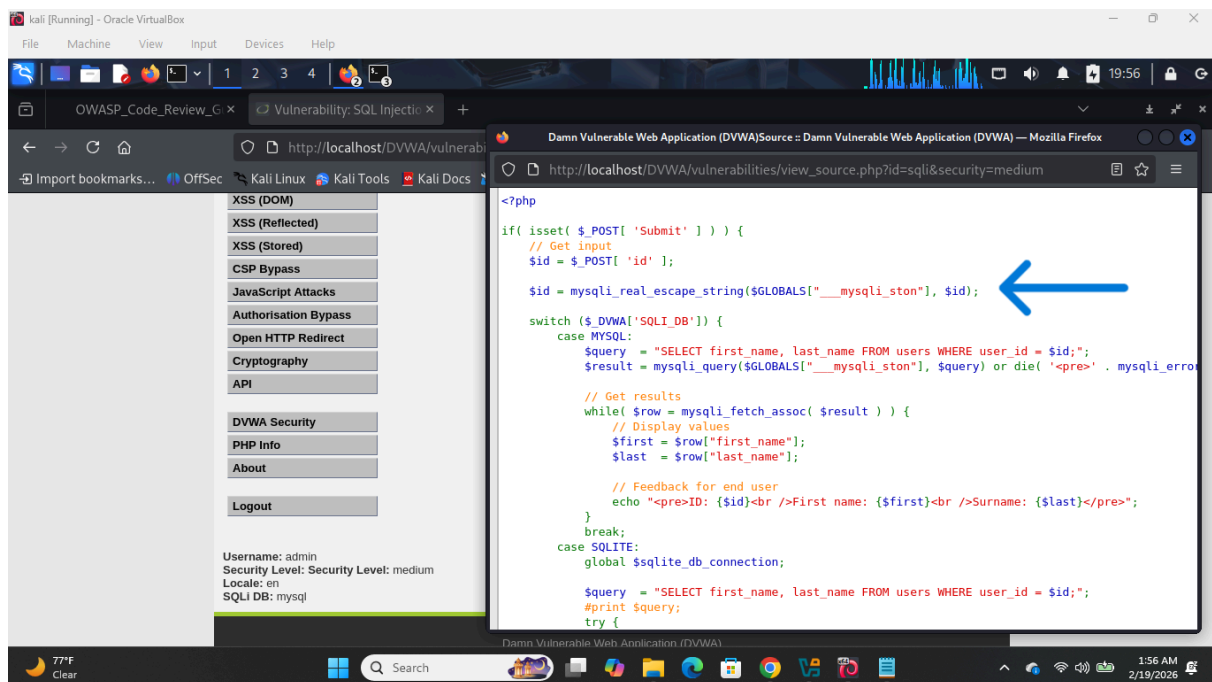
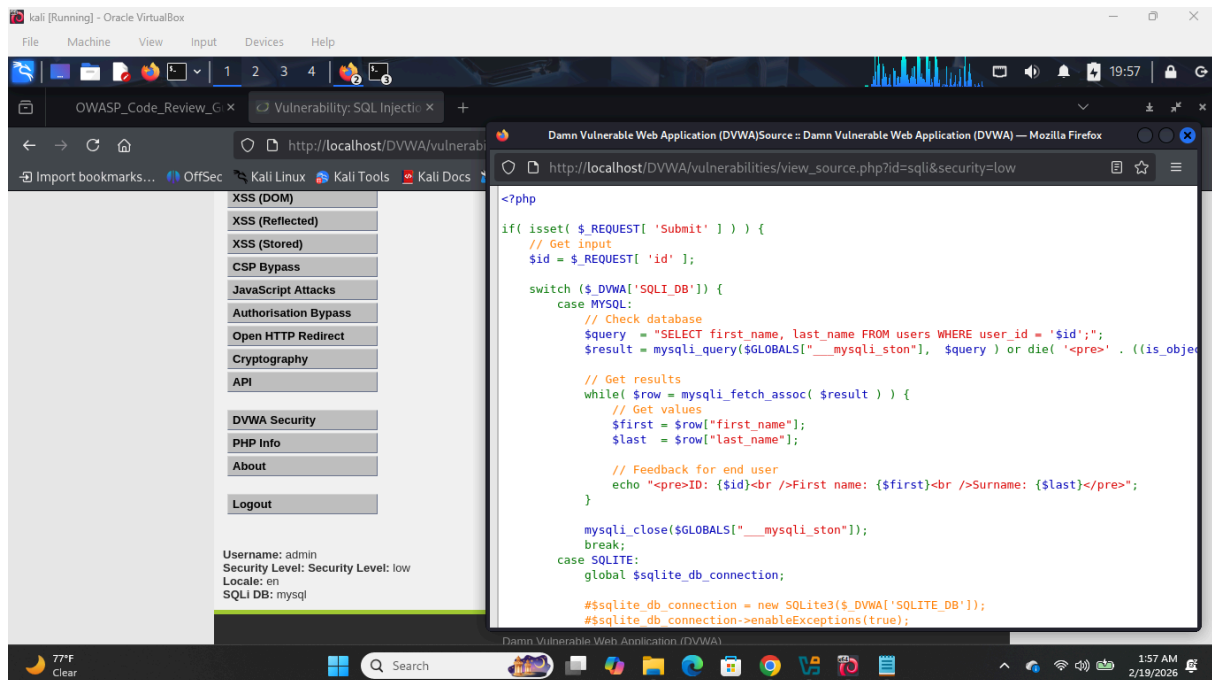
Understanding the Defense

Compared to the "Low" security level, the "Medium" level has an extra layer of defense in its code.

In "Low" level, the "id" parameter is directly used in the query without any sanitization or escaping.

The “medium” level uses the “mysql_real_escape_string” function to escape the string literals.

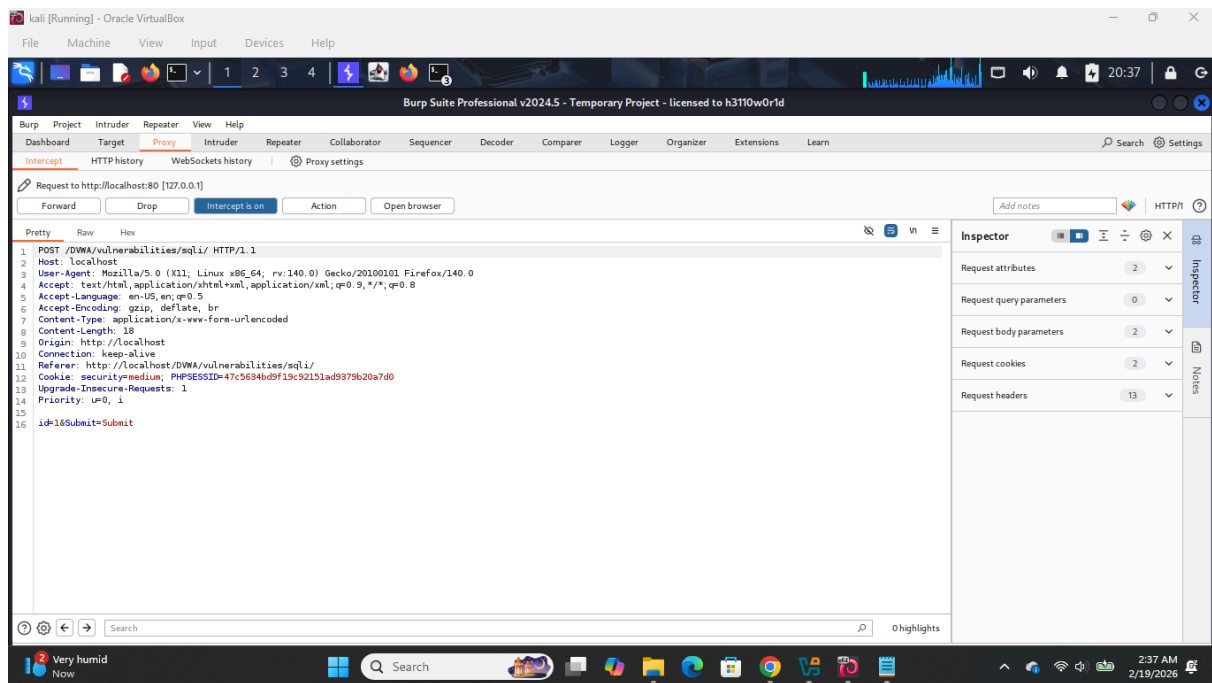
However, this extra layer still has it’s vulnerabilities leaving the app vulnerable to SQLi.



Exploitation (Discovery Phase)

Using my "Burpsuite", I intercepted the request from the app.

Here I identified an input field (id). This will be tested to exploit the potential SQLi vulnerability in the web app.

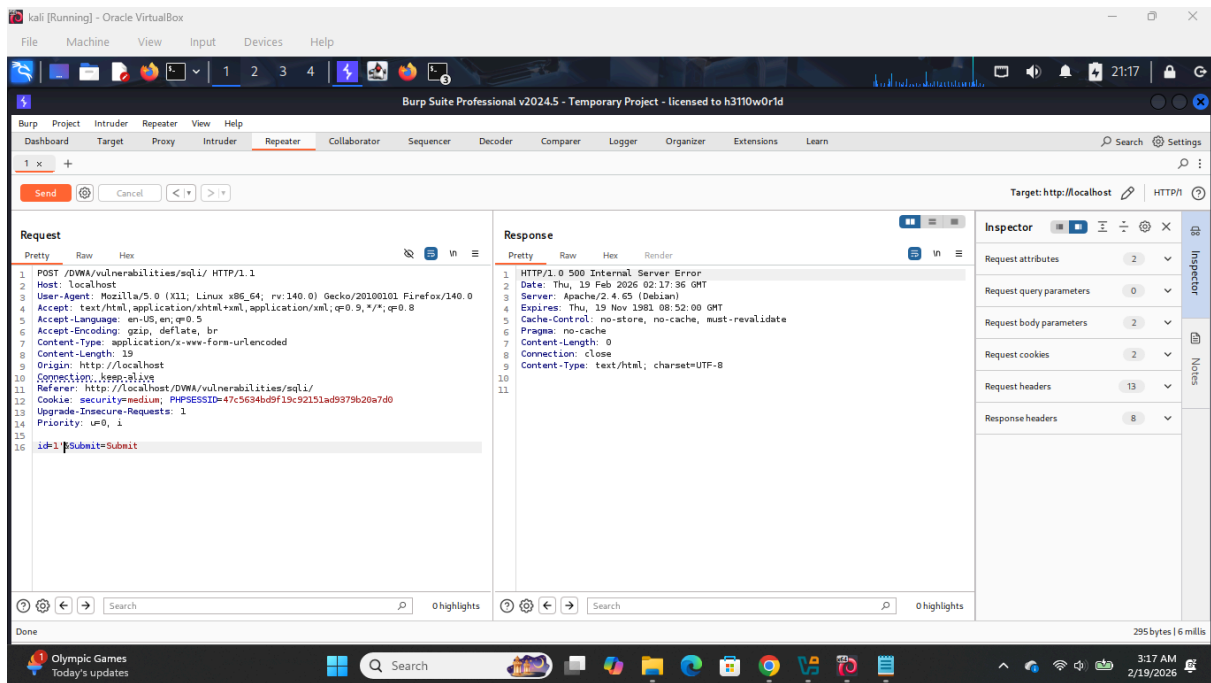


To determine if the app is vulnerable to SQLi, I tried adding malicious payload to the "id" parameter to see how the app reacts.

I tried a few like

1' 1" 1' OR 1=1 1' OR '1'='1

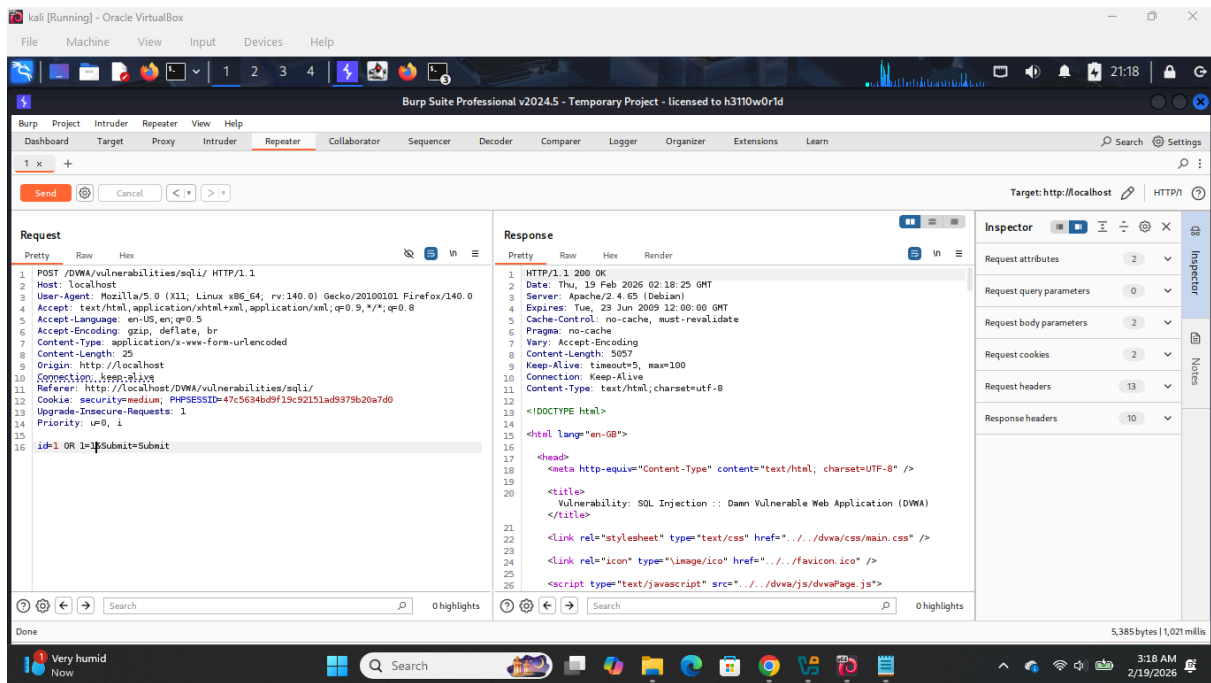
I received a "Server error" in return. These payloads failed because of the "mysqli_real_escape_string" function.



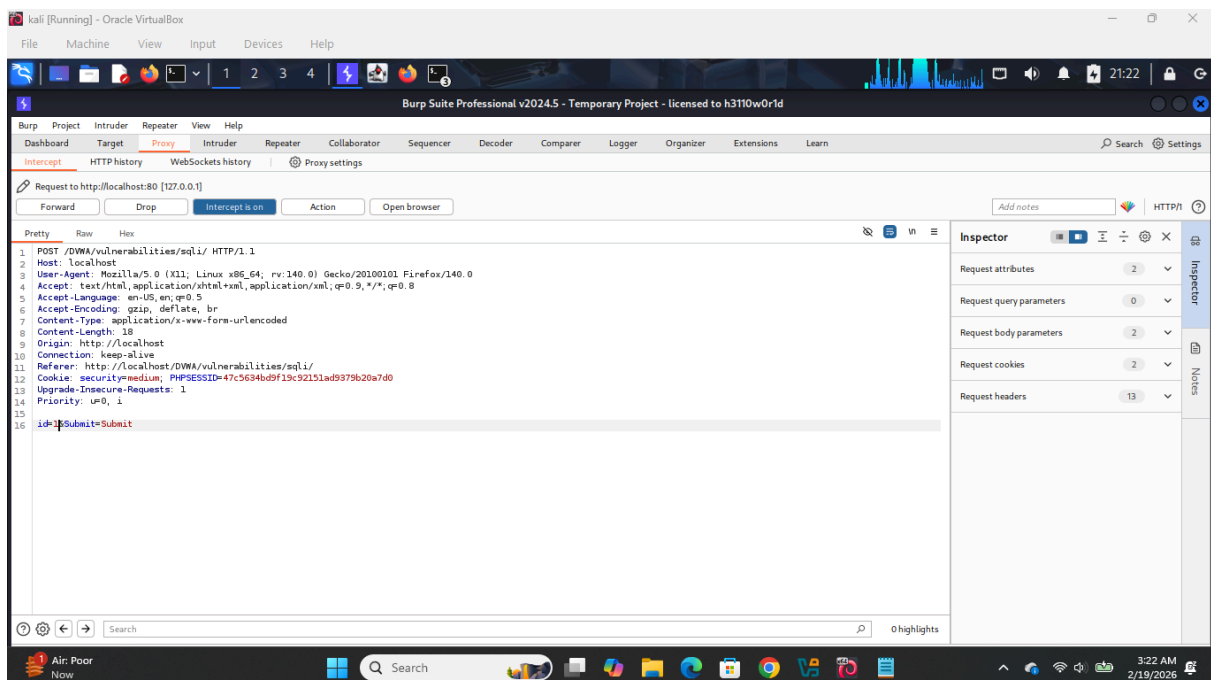
Explanation of the successful bypass

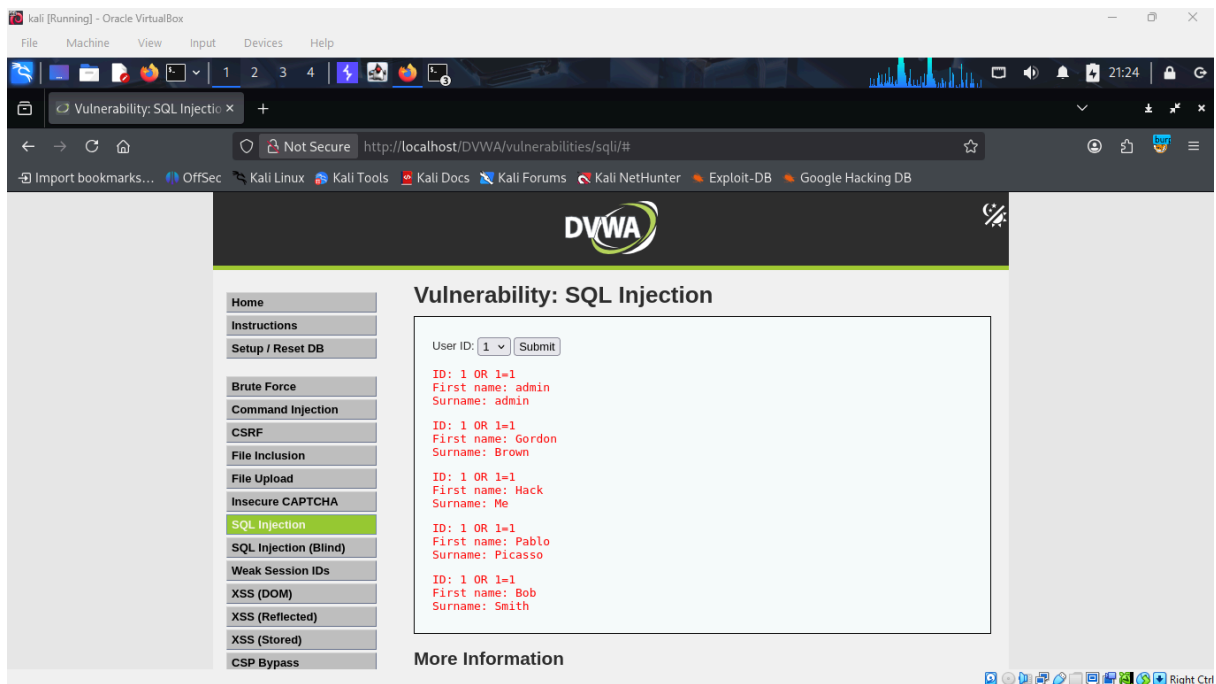
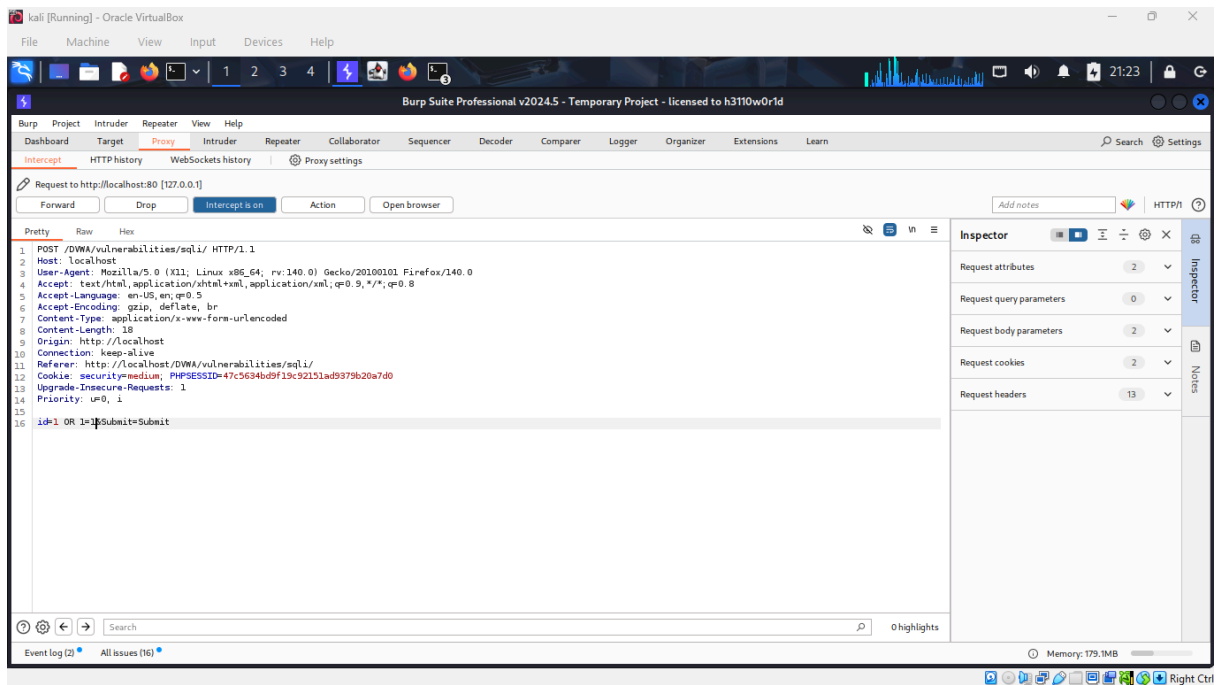
After my few failed attempts, I successfully bypassed the "mysqli_real_escape_string" function by using the payload (1 or 1=1). This payload bypasses the function because there are no quotes.

The function sees quotes as dangerous which is why previous payloads failed.



After successfully bypassing the escape function, I used my burpsuite to intercept and modify request from the app. This payload (1 OR 1=1) breaks the SQL query, resulting to the database returning the information of every user's record on the database.





Defense and Remediation

The Problem:

- The query looks like "...WHERE id = \$id", the function is almost useless. An attacker can still inject SQL commands because there are no quotes to

"break out of."

- **"mysqli_real_escape_string"** only works if the variable you are escaping is **wrapped in single quotes** inside your SQL query.
- It Only Protects "Strings". The function is designed specifically to escape string literals.

Remediation

- **Parameterized queries or Prepared Statements:** Instead of manually escaping strings, you should use Prepared Statements. This is the industry standard because it separates the SQL logic from the data. The database compiles the query before it ever sees your data. The data is treated strictly as a value, never as code. You specify if the data is a string ("s"), integer ("i"), or double ("d"). Your code is much cleaner without all the concatenation and manual escaping.
- **Input handling best practices:** Input handling involves checking and cleaning data before it ever touches a database query. While it won't stop a complex SQLi on its own, it is essential for overall security. The best practices include
 - > **whitelisting** where you only define what is "good."
 - > **Datatype validation** using functions like `is_numeric()` or cast it to an integer `(int)$id`
 - > **Sanitization:** Removing or "stripping" dangerous characters (like removing `<script>` tags). This practice can break legitimate data (like the name "O'Reilly").
 - > **Encoding:** Converting characters into a safe format (like changing `'` to `\'` or `&` to `&`).

References

Burpsuite; Proxy intercept, Repeater

DVWA; SQL Injection Lab

Project 3: Building a Secure Login Module After a SQL Injection Incident

source code file

[183P-K3rceMQCKqx27Sj3yB24hH4ktP1A?usp=sharing](https://drive.google.com/drive/folders/183P-K3rceMQCKqx27Sj3yB24hH4ktP1A?usp=sharing)

<https://drive.google.com/drive/folders/183P-K3rceMQCKqx27Sj3yB24hH4ktP1A?usp=sharing>

Why this is secure:

1. **Separation of Concerns:** By using the `?` placeholder, the database driver ensures that the structure of the SQL command is fixed before the user data is added.
2. **No Manual Escaping:** It avoids unreliable functions like `mysql_real_escape_string`. The database driver handles all necessary character handling automatically at the protocol level.

Example usage:

Even if a malicious input is submitted, the query will safely fail to find a user rather than executing the '1 OR 1=1' command.

```
malicious_input = "1 OR 1=1"
get_user_info(malicious_input)
```

How the Secure Code Handles It;

In the provided Python backend, the query is defined as `SELECT ... WHERE id = ?`. When `cursor.execute(query, (id,))` is called, the database driver performs the following:

- **Literal Interpretation:** The database engine has already "compiled" the `SELECT` statement. It treats the entire string `"1 OR 1=1"` as a single, literal value for the `id` field.
- **No Command Execution:** Because the data is separated from the logic, the database does not see the semicolon (`=`) as a command terminator. It

simply looks for a user whose ID is literally the string `"1 OR 1=1"`.

- **Safe Failure:** Since no user has such a strange ID, the query returns no results. The `1=1` BOOLEAN value is never executed because it was never parsed as a command by the database.

Comparison with the Vulnerable PHP Code

If this same input were passed to the PHP code you provided:

1. The code would concatenate the string: `SELECT ... WHERE id = 1 OR 1=1;`.
2. The `mysqli_real_escape_string` function would not stop this because there are no quotes to escape in that specific payload.
3. The database would execute the first command (selecting user 1) and then immediately execute the second command, listing the entire table.

References

Google Gemini AI;

Prompt;

The php code in the file is a backend function that handles sql queries.
write a secure backend using python in place of php that:

- Accepts user input (e.g. user ID)
- Queries a database for user information
- Prevents SQL injection completely
- Works even if an attacker submits malicious input

dvwaphp

 PHP



The php code in the file is a backend function that handles sql queries.



write a secure backend using python in place of php that:

- Accepts user input (e.g. user ID)...